

全国计算机二级公共基础知识(重点部分)

第一章 数据结构基础

1.1 算法

1.1.1 算法的基本概念

算法是解题方案的准确而完整的描述,它不等于程序,也不等计算方法。

算法的基本特征

可行性(effectiveness)

确定性(definiteness)

有穷性(finiteness)

拥有足够的情报

算法的时间复杂度

执行算法所需要的计算工作量

与下列因素有关:

书写算法的程序设计语言,编译产生的机器语言,代码质量

机器执行指令的速度,问题的规模

问题的规模函数 算法的工作量= $f(n)$

算法中基本操作重复执行的频率 $T(n)$, 是问题规模 n 的某个函数 $f(n)$, 记作: $T(n)=O(f(n))$

记号“ O ”读作“大 O ”。表示随问题规模 n 的增加, 算法执行时间的增长率和 $f(n)$ 相应增加。

常见算法复杂度:

$O(1)$: 常数阶 $O(n)$: 作线性阶 $O(n^2)$: 平方阶

$O(n^3)$: 立方阶 $O(\log n)$: 对数阶 $O(2^n)$: 指数阶

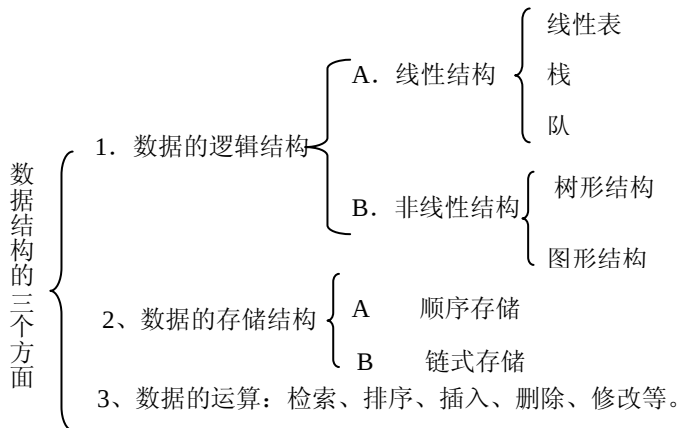
算法的空间复杂度

算法执行过程中所需的最大存储空间

存储量包括以下三部分

算法程序所占的空间,输入的初始数据所占的存储空间,算法执行过程中所要的额外空间

1.2 数据结构的基本概念



数据的逻辑结构

对数据元素之间的逻辑关系的描述

只抽象地反映数据元素之间的逻辑关系,与计算机中的存储无关

数据的存储结构

数据的逻辑结构在计算机存储空间中的存放形式

常用的存储结构: 顺序, 链式, 索引

一种数据结构可根据需要采用不同的存储结构。采用不同的存储结构, 其数据处理的效率是不同

线性结构

如果一个非空数据结构满足下列两个条件:

有且只有一个根结点;

每一个结点最多有一个前件, 也最多有一个后件。

常见的线性结构有: 线性表、栈与队列、线性链表

非线性结构

如果一个数据结构不是线性结构
常见的非线性结构有：树、二叉树、图

1.3 线性表及其顺序存储结构

线性表的结构特征

数据元素在表中的位置由序号决定，数据元素之间的相对位置是线性的；

对于一个非空线性表，有且只有一个根结点 a_1 ，它无前件，有且只有一个终端结点 a_n ，它无后件，除根结点与终端结点外，其他所有结点有且只有一个前件，也有且只有一个后件。

线性表的存储结构

顺序存储 链式存储

两个基本特点：

线性表中所有元素所占的存储空间是连续的。

线性表中各数据元素在存储空间中是按逻辑顺序依次存放的。

存储示意图：

逻辑地址	数据元素	物理地址
	$\vdots$	
1	a_1	$\text{Loc}(a_1)$
2	a_2	$\text{Loc}(a_1) + k$
	$\vdots$	
i	a_i	$\text{Loc}(a_1) + (i-1)k$
	$\vdots$	
n	a_n	$\text{Loc}(a_1) + (n-1)k$
	$\vdots$	

顺序表的插入运算

V(1..10)		
1	19	
2	30	
3	55	
4	62	
5	38	
6	21	
7	36	
8	67	
9		
10		

(a) 长度为8的线性表

V(1..10)		
1	19	
2	83	
3	30	
4	55	
5	62	
6	38	
7	21	
8	36	
9	67	
10		

(b) 插入83后，长度变为9

V(1..10)		
1	19	
2	83	
3	30	
4	55	
5	62	
6	38	
7	12	
8	21	
9	36	
10	67	

(c) 插入12后，长度变为10

顺序表的删除运算

V(1..10)		
1	19	
2	30	
3	55	
4	62	
5	38	
6	21	
7	36	
8	67	
9		
10		

(a) 长度为8的线性表

V(1..10)		
1	30	
2	55	
3	62	
4	38	
5	21	
6	36	
7	67	
8		
9		
10		

(b) 删除19后，长度变为7

V(1..10)		
1	30	
2	55	
3	62	
4	38	
5	21	
6	67	
7		
8		
9		
10		

(c) 删除36后，长度变为6

插入算法的分析

假设线性表中含有 n 个数据元素，在进行插入操作时，若假定在 $n+1$ 个位置上插入元素的可能性均等，则平均移动元素的个数为：

$$E_{is} = \frac{1}{n+1} \sum_{i=1}^{n+1} p_i (n-i+1) = \frac{n}{2}$$

删除算法的分析

在进行删除操作时，若假定删除每个元素的可能性均等，则平均移动元素的个数为：

$$E_{dl} = \frac{1}{n} \sum_{i=1}^n p_i (n-i) = \frac{n-1}{2}$$

分析结论

顺序存储结构表示的线性表，在做插入或删除操作时，平均需要移动大约一半的数据元素。当线性表的数据元素量较大，并且经常要对其做插入或删除操作时，这一点需要值得考虑

1.4 栈和队列

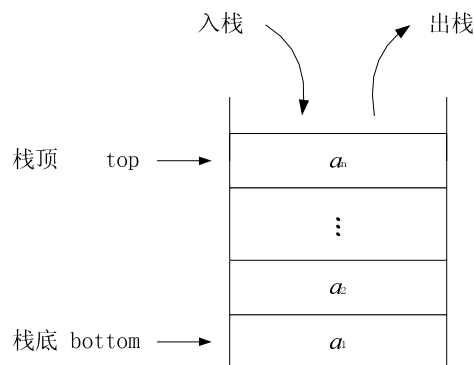
栈的定义

栈 (stack)：一种只允许在表的一端进行插入或删除操作的特殊的线性表

栈顶(top)：允许进行插入与删除操作的一端

栈底(bottom)：不允许插入与删除操作的另一端

先进后出 (FILO) 或后进先出(LIFO)的线性表

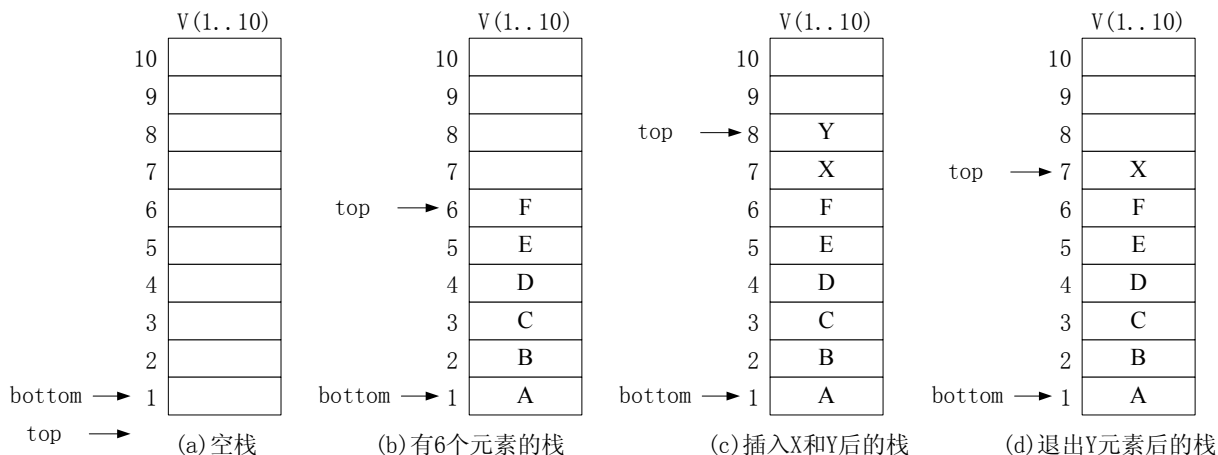


栈的顺序存储及其运算

top=0：栈空

top=m：栈满

栈的基本运算 入栈运算， 退栈运算， 读栈顶元素



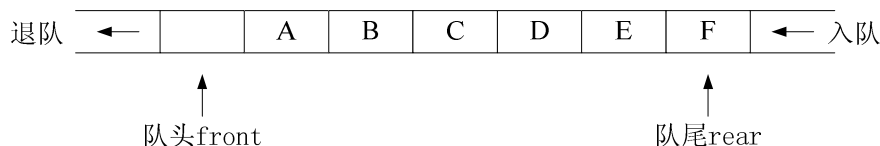
队列的定义

限定只能在表的一端进行插入和在另一端进行删除操作的线性表

队尾(rear)：允许插入的一端

队头(front)：允许删除的另一端

先进先出 (FIFO) 表或后进后出 (LILO) 线性表



基本操作

入队运算：往队列的队尾插入一个元素，队尾指针 **rear** 的变化

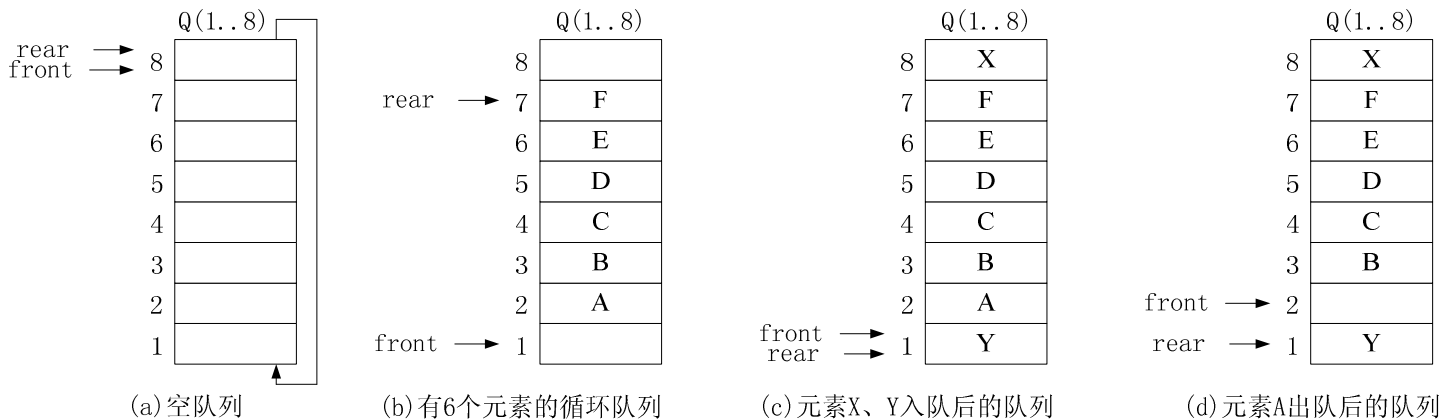
退队运算：从队列的排头删除一个元素，队头指针 **front** 的变化

循环队列及其运算

队列存储空间最后一个位置绕到第一个位置，形成逻辑上的环状空间，供队列循环使用

入队运算：队尾指针加 1，并当 $rear=m+1$ 时置 $rear=1$

出队运算：队头指针加 1，并当 $front=m+1$ 时置 $front=1$



1.5 线性链表

线性表顺序存储的缺点

插入或删除的运算效率很低。在顺序存储的线性表中，插入或删除数据元素时需要移动大量的数据元素。

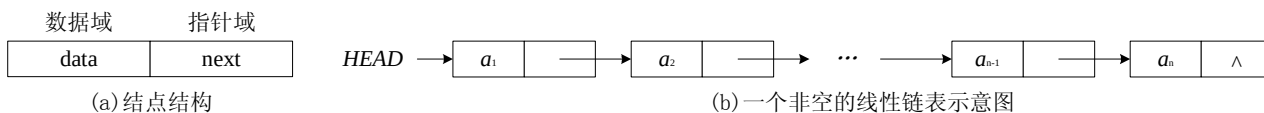
线性表的顺序存储结构下，线性表的存储空间不便于扩充。

线性表的顺序存储结构不便于对存储空间的动态分配。

线性表的链式存储结构

物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接来实现的

每个结点由两部分组成：数据域和指针域



与顺序存储相比，链表的优点有：

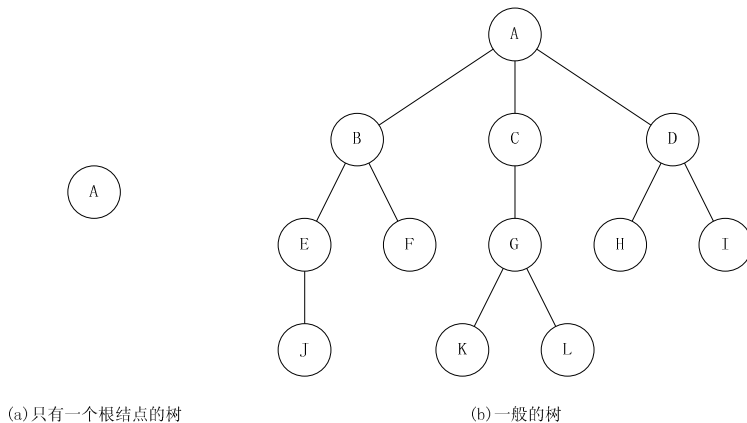
插入和删除元素时，不需要移动数据元素，只需要修改指针即可

1.6 树与二叉树

树的定义

树(Tree)是 $n(n \geq 0)$ 个结点的有限集 T ， T 为空时称为空树，否则它满足如下两个条件：

- (1) 有且仅有一个特定的称为根(Root)的结点；
- (2) 其余的结点可分为 $m(m \geq 0)$ 个互不相交的子集 $T_1, T_2, T_3, \dots, T_m$ ，其中每个子集又是一棵树，并称其为子树。



树的基本概念

父结点与树的根：每个结点最多只允许有一个前件，称为该结点的父结点。没有前件的结点中有一个，称为树的根结点。

子结点与叶子结点：在树结构中，每一个结点可以有多个后件，它们都称为该结点的子结点。没有后件的结点称为叶子结点。

结点的度和树的度：一个结点所拥有后件个数称为该结点的度。一棵树中各个结点度数的最大值叫做这棵树的度。

层和树的深度：树结构是一种层次结构，根结点为第一层，根的子结点为第二层，其余各结点的层数逐层由上而下计算。一棵树中结点的最大层数叫做此树的深度。

树的特点

- (1) 树中只有根结点没有前件；
- (2) 除根外，其余结点都有且仅一个前件；
- (3) 树的结点，可以有零个或多个后件；
- (4) 除根外的其他结点，都存在唯一一条从根到该结点的路径；
- (5) 树是一种分支结构（除根的结点外）每个元素都有且仅有一个直接前件，有且仅有零个或多个直接后件。

二叉树及其基本性质

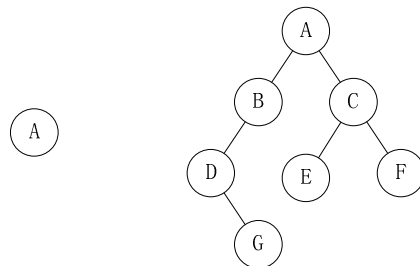
二叉树的定义

一个二叉树是 n 个结点的有限集合 ($n \geq 0$)，此集合或者是空集 ($n=0$)，或者是由一个根结点及两棵互不相交的、分别称为左子树和右子树的二叉树组成，并且左右子树都是二叉树。

特点：

非空二叉树只有一个根结点；

每一个结点最多有两棵子树，且分别称为该结点的左子树与右子树。



(a) 只有一个根结点的二叉树 (b) 深度为4的二叉树

二叉树的性质

性质 1 在二叉树的第 k 层上，最多有 $2^{k-1} (k \geq 1)$ 个结点。

性质 2 深度为 m 的二叉树最多有个 $2^m - 1$ 结点

性质 3 在任意一棵二叉树中，度数为 0 的结点（即叶子结点）总比度为 2 的结点多一个。即：

$$n_0 = n_2 + 1$$

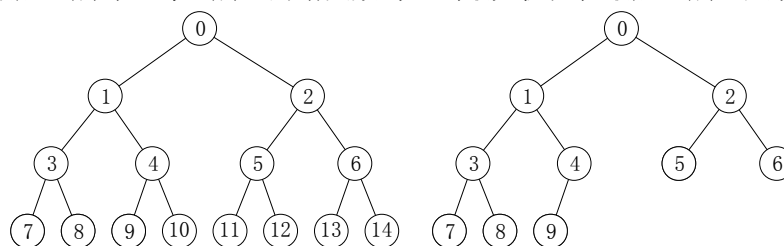
其中， n_0 表示度数为 0 的结点数， n_2 表示度数为 2 的结点数

性质 4 具有 n 个结点的二叉树的深度至少为 $\lceil \log_2 n \rceil + 1$ ，其中 $\lceil \log_2 n \rceil$ 表示取 $\log_2 n$ 的整数部分。

满二叉树和完全二叉树

满二叉树：除最后一层外，每一层上的所有结点都有两个子结点。

完全二叉树：除最后一层外，每一层上的结点数均达到最大值；在最后一层上只缺少右边的若干结点。



(a) 满二叉树

(b) 完全二叉树

性质 5 具有 n 个结点的完全二叉树深度为 $\lceil \log_2 n \rceil + 1$

二叉树的遍历：不重复地访问二叉树中的所有结点

1. 前序遍历（DLR）

首先访问根结点，然后遍历左子树，最后遍历右子树；并且，在遍历左右子树时，仍然先访问根结点，然后遍历左子树，最后遍历右子树。

2. 中序遍历（LDR）

首先遍历左子树，然后访问根结点，最后遍历右子树；并且，在遍历左、右子树时，仍然先遍历左子树，然后访问根结点，最后遍历右子树

3. 后序遍历（LRD）

首先遍历左子树，然后遍历右子树，最后访问根结点，并且，在遍历左、右子树时，仍然先遍历左子树，然后遍历右子树，最后访问根结点。

前序遍历：

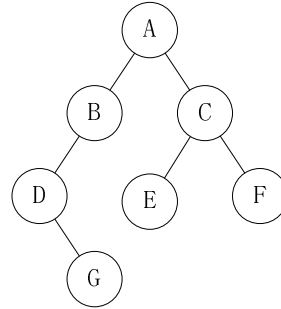
A、B、D、G、C、E、F

中序遍历：

D、G、B、A、E、C、F

后序遍历：

G、D、B、E、F、C、A



1.7 查找技术

查找（Searching）：根据给定的某个值，在查找表中确定一个其关键字等于给定值的数据元素。

查找结果：

查找成功：找到

查找不成功：没找到

平均查找长度：查找过程中关键字和给定值比较的平均次数

顺序查找

基本思想：

从表中的第一个元素开始，将给定的值与表中逐个元素的关键字进行比较，直到两者相符，查到所要找的元素为止。否则就是表中没有要找的元素，查找不成功。

平均要与表中一半以上元素进行比较，最坏情况下需要比较 n 次。

下列两种情况下只能采用顺序查找：

如果线性表是无序表（即表中的元素是无序的），则不管是顺序存储结构还是链式存储结构，都只能用顺序查找。

即使是有序线性表，如果采用链式存储结构，也只能用顺序查找。

二分法查找

思想：先确定待查找记录所在的范围，然后逐步缩小范围，直到找到或确认找不到该记录为止。

前提：必须在具有顺序存储结构的有序表中进行。

查找过程：

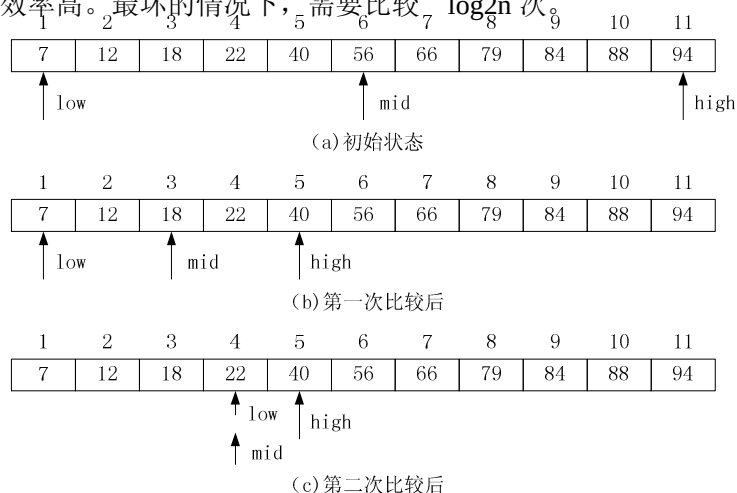
若中间项的值等于 x ，则说明已查到。

若 x 小于中间项的值，则在线性表的前半部分查找；

若 x 大于中间项的值，则在线性表的后半部分查找。

特点：比顺序查找方法效率高。最坏的情况下，需要比较 $\log_2 n$ 次。

例：查找元素 22 过程：



1.8 排序技术

冒泡排序

基本思想

对存放原始数据的数组，按从后往前的方向进行多次扫描，当发现相邻两个数据的次序与排序要求的“递增次序”不符合时，即将这两个数据进行互换。这样，较小的数据就会逐单元向前移动，好象气泡向上浮起一样。

性能分析

假设线性表的长度 n ，则在最坏情况下，需要的比较次数为 $n(n-1)/2$ 。

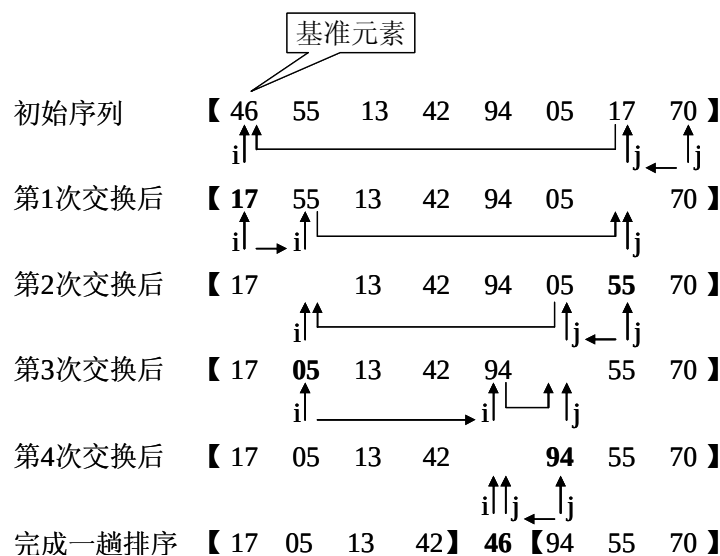
49	13	13	13	13	13	13	13
38	49	27	27	27	27	27	27
65	38	49	38	38	38	38	38
97	65	38	49	49	49	49	49
76	97	65	49	49	49	49	49
13	76	97	65	65	65	65	65
27	27	76	97	76	76	76	76
49	49	49	76	97	97	97	97
初始序列	第一趟排序后	第二趟排序后	第三趟排序后	第四趟排序后	第五趟排序后	第六趟排序后	第七趟排序后

快速排序

基本思想

任取待排序序列中的某个元素作为基准（一般取第一个元素），通过一趟排序，将待排元素分为左右两个子序列，左子序列元素的排序码均小于或等于基准元素的排序码，右子序列的排序码则大于基准元素的排序码，然后分别对两个子序列继续进行排序，直至整个序列有序。

快速排序的平均时间复杂度为 $O(n\log_2 n)$ 。



插入类排序法

基本思想：

每次将一个待排序的记录，按其关键字大小插入到前面已经排好序的子序列中的适当位置，直到全部记录插入完成为止。

方法:

简单插入排序

简单插入排序法

基本思想:

把 n 个待排序的元素看成为一个有序表和一个无序表, 开始时有序表中只包含一个元素, 无序表中包含有 $n-1$ 个元素, 排序过程中每次从无序表中取出第一个元素, 把它的排序码依次与有序表元素的排序码进行比较, 将它插入到有序表中的适当位置, 使之成为新的有序表。

在最坏的情况下, 需要 $n(n-1)/2$ 次比较。

初始序列		【49】	38	65	97	76	13	27	<u>49</u>
第1趟排序	(38)	【38	49】	65	97	76	13	27	<u>49</u>
第2趟排序	(65)	【38	49	65】	97	76	13	27	<u>49</u>
第3趟排序	(97)	【38	49	65	97】	76	13	27	<u>49</u>
第4趟排序	(76)	【38	49	65	76	97】	13	27	<u>49</u>
第5趟排序	(13)	【13	38	49	65	76	97】	27	<u>49</u>
第6趟排序	(27)	【13	27	38	49	65	76	97】	<u>49</u>
第7趟排序	(49)	【13	27	38	49	<u>49</u>	65	76	97】

简单选择排序法

基本思想:

扫描整个线性表, 从中选出最小的元素, 将它交换到表的最前面; 然后对剩下的子表采用同样的方法, 直到子表空为止。

最坏情况下, 需要比较 $n(n-1)/2$ 次。

初始序列	【49	38	65	97	76	13	27	<u>49</u> 】
第1趟排序	13	【38	65	97	76	49	27	<u>49</u> 】
第2趟排序	13	27	【65	97	76	49	38	<u>49</u> 】
第3趟排序	13	27	38	【97	76	49	65	<u>49</u> 】
第4趟排序	13	27	38	49	【76	97	65	<u>49</u> 】
第5趟排序	13	27	38	49	<u>49</u>	【97	65	76】
第6趟排序	13	27	38	49	<u>49</u>	65	【97	76】
第7趟排序	13	27	38	49	<u>49</u>	65	76	【97】

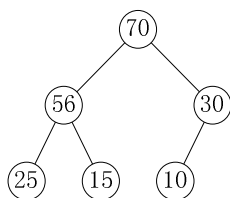
堆排序法

堆的定义

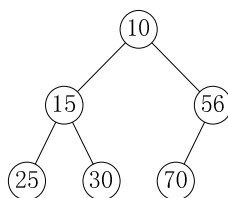
具有 n 个元素的序列, 当且仅当满足

$$\begin{cases} h_i \geq h_{2i} \\ h_i \geq h_{2i+1} \end{cases} \quad \text{① 或} \quad \begin{cases} h_i \leq h_{2i} \\ h_i \leq h_{2i+1} \end{cases} \quad \text{②}$$

时称之为堆。①称为大根堆; ②称为小根堆。



(a) 大根堆



(b) 小根堆

建堆

在建堆的过程中，总是将根结点值与左、右子树的根结点值进行比较，若不满足堆的条件，则将左、右子树根结点值中的大者与根结点值进行交换。这个调整过程一直做到所有子树均为堆为止。

堆排序

(1) 首先将一个无序序列建成堆。

(2) 然后将堆顶元素(序列中的最大项)与堆中最后一个元素交换(最大项应该在序列的最后)。不考虑已经换到最后的那个元素，只考虑前 $n-1$ 个元素构成的子序列，将该子序列调整为堆。

反复做步骤 (2)，直到剩下的子序列空为止。

在最坏情况下，堆排序法需要比较的次数为 $O(n\log_2 n)$ 。

各种排序法比较

类别	排序方法	基本思想	时间复杂度
交换类	冒泡排序	相邻元素比较，不满足条件时交换	$n(n-1)/2$
	快速排序	选择基准元素，通过交换，划分成两个子序列	$O(n\log_2 n)$
插入类	简单插入排序	待排序的元素看成为一个有序表和一个无序表，将无序表中元素插入到有序表中	$n(n-1)/2$
	希尔排序	分割成若干个子序列分别进行直接插入排序	$O(n^{1.5})$
选择类	简单选择排序	扫描整个线性表，从中选出最小的元素，将它交换到表的最前面	$n(n-1)/2$
	堆排序	选建堆，然后将堆顶元素与堆中最后一个元素交换，再调整为堆	$O(n\log_2 n)$

第 2 章 程序设计基础

2.1 程序设计方法与风格

结构化设计方法

模块内部程序各部分要按照自顶向下的结构划分

各程序部分应按功能组合

各程序之间的联系尽量通过调用子程序来实现，不用或少用 GOTO 方式

面向对象程序设计方法

程序设计风格

原则：清晰第一，效率第二

1. 源程序中的内部文档

符号名的命名：有一定实际含义

程序的注释：

序言性注释，功能性注释

程序的视觉组织：层次清晰

2. 数据说明

数据说明的次序规范化，说明语句中变量安排有序化，使用注释来说明复杂数据的结构

2.2 结构化程序的基本结构与特点

三种基本结构

顺序结构，选择结构，重复结构

结构化程序设计原则和方法的应用

用有限的控制结构，一个入口和一个出口，每块只有一个入口和一个出口，使用嵌套，前后一致，避免 GOTO 语句

2.3 面向对象的程序设计

主要优点

与人类习惯的思维方法一致，稳定性好，可重用性好，易于开发大型软件产品，可维护性好

对象(Object)

对象是基本的运行时认得实体，它既包括数据（属性），也包括作用于数据的操作（行为）。

一个对象把属性和行为封装为一个整体

一个对象通常可由对象名、属性和操作 3 部分组成

对象特点

标识惟一性 ,分类性 ,多态性 ,封装性 ,模块独立性好

类和实例

类是具有共同属性、共同操作方法的对象的集合，是对象的抽象

对象是其对应类的一个实例

消息

对象之间进行通信的机制

三部分组成

接收消息的对象名称 ,消息标识符（消息名）,零个或多个参数

继承

继承是父类和子类之间共享数据的方法的机制

一个子类可以继承它的父类（或祖先类）中的属性和操作

子类中可以定义自己的属性和操作

单重继承、多重继承

多态性

不同的对象收到同一消息可以产生完全不同的结构，这一现象叫做多态性

优点：灵活性、可重用性、可扩充性。

第 3 章 软件工程基础

3.1 软件工程基本概念

软件的定义和组成

定义：计算机软件（Software）是计算机系统中与硬件相互依赖的另一部分。

组成：程序 ,数据 ,文档

国标(GB)定义：与计算机系统的操作有关的计算机程序、规程、规则，以及可能有的文件、文档及数据。

软件的特点

软件是一种逻辑实体，而不是具体的物理实体，具有抽象性

软件没有明显的制造过程。对软件的质量控制，必须在软件开发方面下功夫

软件不存在老化问题，但存在退化问题，必须要修改和维护

对计算机系统有着依赖性——软件移植的问题

软件复杂性高，开发和维护成本高

软件开发涉及诸多社会因素

软件的分类

应用软件

系统软件：操作系统 ,数据库管理系统 ,设备驱动程序

支撑软件

软件危机

主要表现：

软件需求的增长得不到满足

软件开发成本和进度无法控制

软件质量难以保证

软件不可维护或维护程度非常低

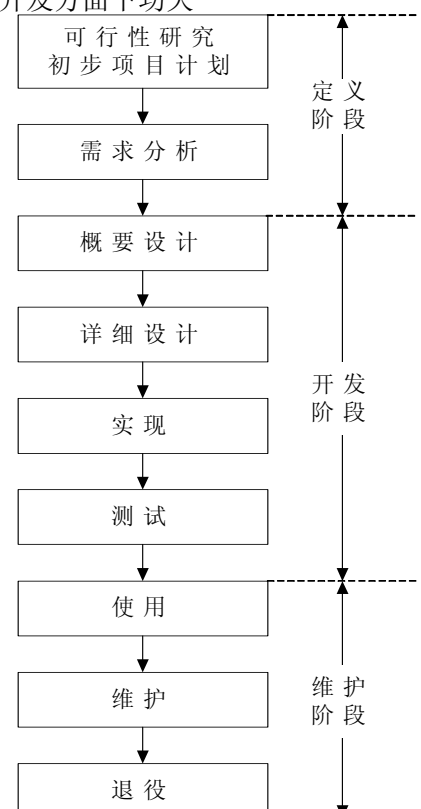
软件成本不断提高

软件开发生产率的提高赶不上硬件的发展和应用需求的增长

软件产品从提出、实现、使用维护、停止使用到退役的过程

3 个阶段 ,6 个阶段工作(右图：)

定义阶段



制定计划：“能做吗？”

需求分析：“做什么？”

开发阶段：

软件设计：“如何做？”，分为概要设计和详细设计两个阶段。

软件实现：“实现”，编码。

软件测试：“做的怎么样？”

运行维护阶段

使用，不断维护

3.2 结构化分析方法

需求分析

定义：

任务：导出目标系统的逻辑模型，解决“做什么”的问题，全面理解用户的各项要求，准确地表达各项要求

主要工作：需求获取，需求分析，编写需求规格说明书，需求审评

需求分析方法

结构化分析方法

面向数据流的结构化分析方法（SA）

面向数据结构的 Jackson 方法（JSD）

关于结构化分析方法

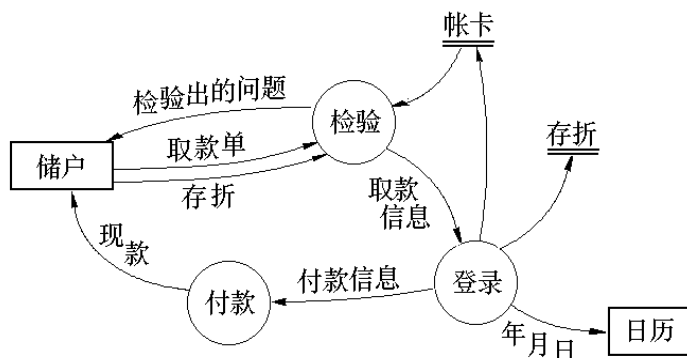
结构化程序设计理论在需求分析阶段的运用，面向数据流进行需求分析的方法，自顶向下、逐层分解

主要工具：数据流图、数据字典

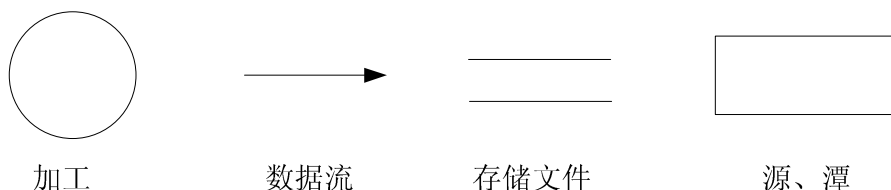
结构化分析的常用工具

数据流图（DFD），数据字典，判定树，判定表

数据流图



数据流图：基本图形元素



软件需求规格说明书

需求分析阶段的最后成果

作用：便于用户、开发人员进行理解和交流；

反映出用户问题的结构，可以作为软件开发工作的基础和依据；

作为确认测试和验收的依据。

主要内容：概述、数据描述、功能描述、性能描述、参考文献、附录

特点：①正确性；②无歧义性；③完整性；④可验证性；⑤一致性；⑥可理解性；⑦可修改性；⑧可追踪性。

3.3 结构化设计方法

内聚性

度量一个模块功能强度的一个相对指标。一个模块只做一件事

耦合性

度量模块之间的相互联系程度

取决于接口的复杂程度、调用方式、哪些信息通过接口

设计准则

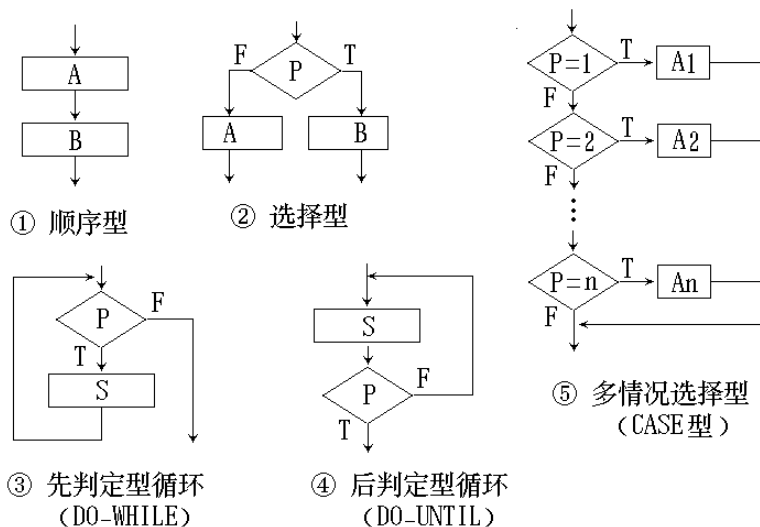
提高模块独立性,深度、宽度、扇度和扇出适度,使模块的作用域在该模块的控制域内,应减少模块的接口和界面的复杂性,设计成单入口、单出口的模块,设计功能可预测的模块

详细设计

程序流程图

图形元素:

方框: 处理步骤



菱形: 逻辑条件

箭头: 控制流

5 种控制结构

顺序型,选择型,先判断重复型,后判断重复型,多分支选择型。

3.4 软件测试

软件测试的目的

检验它是否满足规定的需求或是弄清预期结果与实际结果之间的差别

Grenford J.Myers 观点:

测试是程序的执行过程,目的在于发现错误

一个好的测试用例在于能发现至今未发现的错误

一个成功的测试是发现了至今未发现的错误的测试

静态测试与动态测试

静态测试: 人工评审软件文档或程序,借以发现其中的错误

主要方法: 代码检查、静态结构分析、代码质量度量

动态测试: 上机测试

关键: 设计高效、合理的测试用例

分两类: 白盒测试方法和黑盒测试方法

白盒测试方法与测试用例设计

也称结构测试或逻辑驱动测试,测试用例是根据程序的内部逻辑来设计,主要用于单元测试

逻辑覆盖测试

可分为:

语句覆盖: 每一个语句都能执行一次

路径覆盖: 所有的可能路径都至少经历一次

判定覆盖：每个判定至少都获得一次“真值”和“假值”的机会

条件覆盖：每个判定中每个条件都获得一次“真”和“假”的机会

判断-条件覆盖：判定中的每个条件都能取得各种可能的“真”和“假”值，并且使每个判定都能取到“真”和“假”两种结果

强度顺序：语句覆盖<路径覆盖<判定覆盖<条件覆盖<判定-条件覆盖

黑盒测试方法与测试用例设计

也称功能测试或数据驱动测试

对软件已经实现的功能是否满足需求进行测试和验证

主要方法：等价类划分法，边界值分析法，错误推测法

单元测试

对象：针对程序模块，进行正确性检验的测试

目的：发现各模块内部可能存在的各种差错

依据：从程序的内部结构出发设计测试用例，其依据是详细的设计说明书和源程序

方法：以白盒测试为主，辅以黑盒测试

集成测试

任务：把模块在按照设计要求组装起来的同时进行测试

目的：发现与接口有关的错误

依据：集成测试的依据是概要设计说明书

内容：软件单元的接口测试、全局数据结构测试、边界条件和非法输入的测试

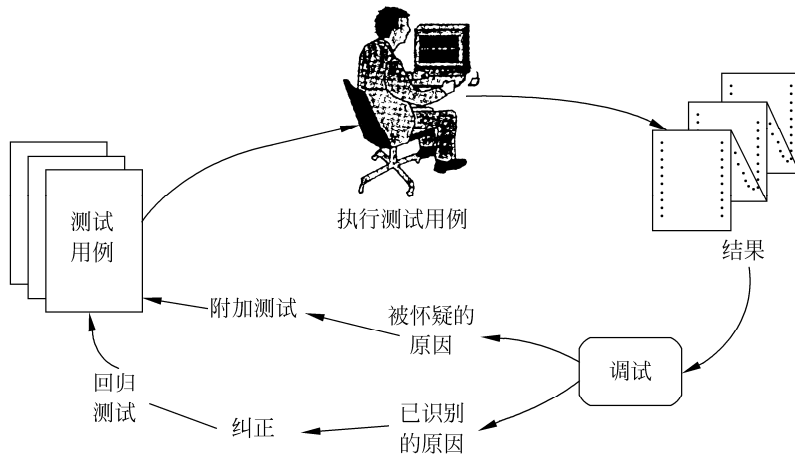
方式：非增量方式组装与增量方式组装。

3.5 程序的调试

任务：诊断和改正程序中的错误

时机：调试主要在开发阶段进行

基本步骤：错误定位、纠正错误、回归测试



第4章 数据库设计基础

4.1 数据库的基本概念

数据是数据库中存储的基本对象，它是描述事物的符号记录。

数据库是长期储存在计算机内、有组织的、可共享的大量数据的集合，它具有统一的结构形式并存放于统一的存储介质内，是多种应用数据的集成，并可被各个应用程序所共享，所以数据库技术的根本目标是解决数据共享问题。

数据库管理系统（DBMS，Database Management System）是数据库的机构，它是一种系统软件，负责数据库中的数据组织、数据操作、数据维护、控制及保护和数据服务等。数据库管理系统是数据系统的核心。为完成数据库管理系统的功能，数据库管理系统提供相应的数据语言：数据定义语言、数据操纵语言、数据控制语言。

4.2 数据库系统的发展和基本特点

数据库系统的发展

数据库管理发展至今已经历了三个阶段：人工管理阶段、文件系统阶段和数据库系统阶段。

数据库系统的基本特点：

- (1) **数据的高集成性。**
- (2) **数据的高共享性与低冗余性。**数据库系统可以减少数据冗余，但无法避免一切冗余。
- (3) **数据独立性。**数据独立性是数据与程序间的互不依赖性，即数据库中数据独立于应用程序而不依赖于应用程序。也就是说，数据的逻辑结构、存储结构与存取方式的改变不会影响应用程序。数据独立性一般分为物理独立性与逻辑独立性两级。
- (4) **数据统一管理与控制。**

4.3 数据库系统的内部结构体系

(1) 数据库系统的三级模式：

概念模式：数据库系统中全局数据逻辑结构的描述，是全体用户（应用）公共数据视图。

外模式：也称子模式或用户模式，它是用户的数据视图，也就是用户所见到的数据模式，它由概念模式推导而出。

内模式：又称物理模式，它给出了数据库物理存储结构与物理存取方法。内模式的物理性主要体现在操作系统及文件级上，它还未深入到设备级上（如磁盘及磁盘操作）。内模式对一般用户是透明的，但它的设计直接影响数据库的性能。

(2) 数据库系统的两级映射：

概念模式/内模式的映射：实现了概念模式到内模式之间的相互转换。当数据库的存储结构发生变化时，通过修改相应的概念模式/内模式的映射，使得数据库的逻辑模式不变，其外模式不变，应用程序不用修改，从而保证数据具有很高的物理独立性。

外模式/概念模式的映射：实现了外模式到概念模式之间的相互转换。当逻辑模式发生变化时，通过修改相应的外模式/逻辑模式映射，使得用户所使用的那部分外模式不变，从而应用程序不必修改，保证数据具有较高的逻辑独立性。

4.4 数据模型

(1) 概念

是数据特征的抽象，它从抽象层次上描述了系统的静态特征、动态行为和约束条件，为数据库系统的信息表示与操作提供一个抽象的框架。

(2) 数据模型的内容

数据模型所描述的内容有三个部分，它们是数据结构、数据操作与数据约束。

(3) 数据模型分类

概念数据模型：简称概念模型，是对客观世界复杂事物的结构描述及它们之间的内在联系的刻画。概念模型主要有：E-R 模型(实体联系模型)。

逻辑数据模型：又称数据模型，是一种面向数据库系统的模型，该模型着重于在数据库系统一级的实现。逻辑数据模型主要有：层次模型、网状模型、关系模型、面向对象模型等。

物理数据模型：又称物理模型，它是一种面向计算机物理表示的模型，此模型给出了数据模型在计算机上物理结构的表示。

(4) 实体联系模型及 E-R 图

E-R 模型的基本概念：

实体：现实世界中的事物。

属性：事物的特性。

联系：现实世界中事物间的关系。实体集的关系有一对一、一对多、多对多的联系。

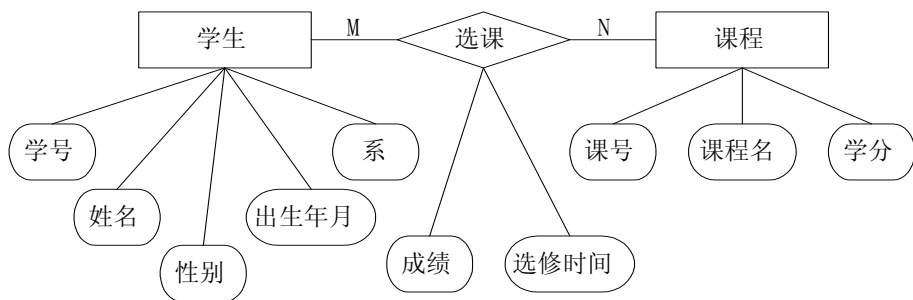
E-R 模型的图示法：

实体集：用矩形表示。

属性：用椭圆形表示。

联系：用菱形表示。

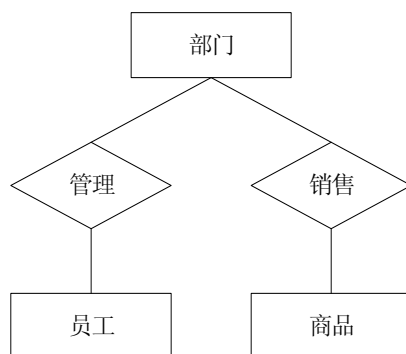
实体集与属性间的联接关系：用无向线段表示。



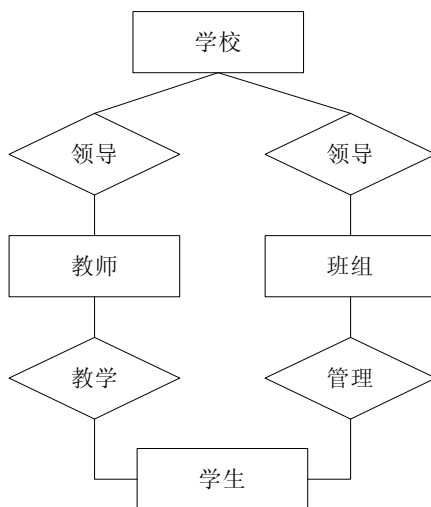
(5) 数据模型

层次模型：具有以下特点：

- ①每棵树有且仅有一个无双亲结点，称为根；
- ②树中除根外所有结点有且仅有一个双亲。



网状模型：是层次模型的一个特例，从图论上看，网状模型是一个不加任何条件限制的无向图。



(a)教学关系E-R图



(b)工作与设备

关系模型：采用二维表来表示，简称表。

(6) 关系中的数据约束：

- ①实体完整性约束
- ②参照完整性约束
- ③用户定义的完整性约束

4.5 关系代数

关系的数据结构

关系是由若干个不同的元组所组成，因此关系可视为元组的集合。 n 元关系是一个 n 元有序组的集合。

关系模型的基本运算：1) 插入；2) 删除；3) 修改；4) 查询（包括投影、选择、笛卡尔积运算）。

关系操纵

关系模型的数据操纵即是建立在关系上的数据操纵，一般有查询、增加、删除和修改四种操作。

集合运算

并 ($\cup$)：关系 R 和 S 具有相同的模式，R 和 S 的并是由属于 R 或属于 S 的元组构成的集合。

差 ($-$)：关系 R 和 S 具有相同的模式，R 和 S 的差是由属于 R 但不属于 S 的元组构成的集合。

交 ($\cap$)：关系 R 和 S 具有相同的模式，R 和 S 的交是由属于 R 且属于 S 的元组构成的集合。

广义笛卡尔积 ($\times$)：设关系 R 和 S 的属性个数分别为 n、m，则 R 和 S 的广义笛卡尔积是一个有 $(n+m)$ 列的元组的集合。每个元组的前 n 列来自 R 的一个元组，后 m 列来自 S

基本的关系

选择：选择指的是从二维关系表的全部记录中，把那些符合指定条件的记录挑出来。

投影：投影是从所有字段中选取一部分字段及其值进行操作，它是一种纵向操作。

联接：联接将两个关系模式拼接成一个更宽的关系模式，生成的新关系中包含满足联接条件的元组。

4.6 数据库设计方法和步骤

需求分析阶段：

数据库设计的第一个阶段，任务主要是收集和分析数据，这一阶段收集到的基础数据 and 数据流图是下一步设计概念结构的基础。

概念设计阶段：

分析数据间内在语义关联，在此基础上建立一个数据的抽象模型，即形成 E-R 图。数据库概念设计的过程包括选择局部应用、视图设计和视图集成。

逻辑设计阶段：

将 E-R 图转换成指定 RDBMS 中的关系模式。

物理设计阶段：

对数据库内部物理结构作调整并选择合理的存取路径，以提高数据库访问速度及有效利用存储空间。